

GyaanSetu Backend & Architecture Deep Dive

A Comprehensive, Code-Verified Architecture & Database Design Master Guide with Visual Diagrams for Placement Preparation

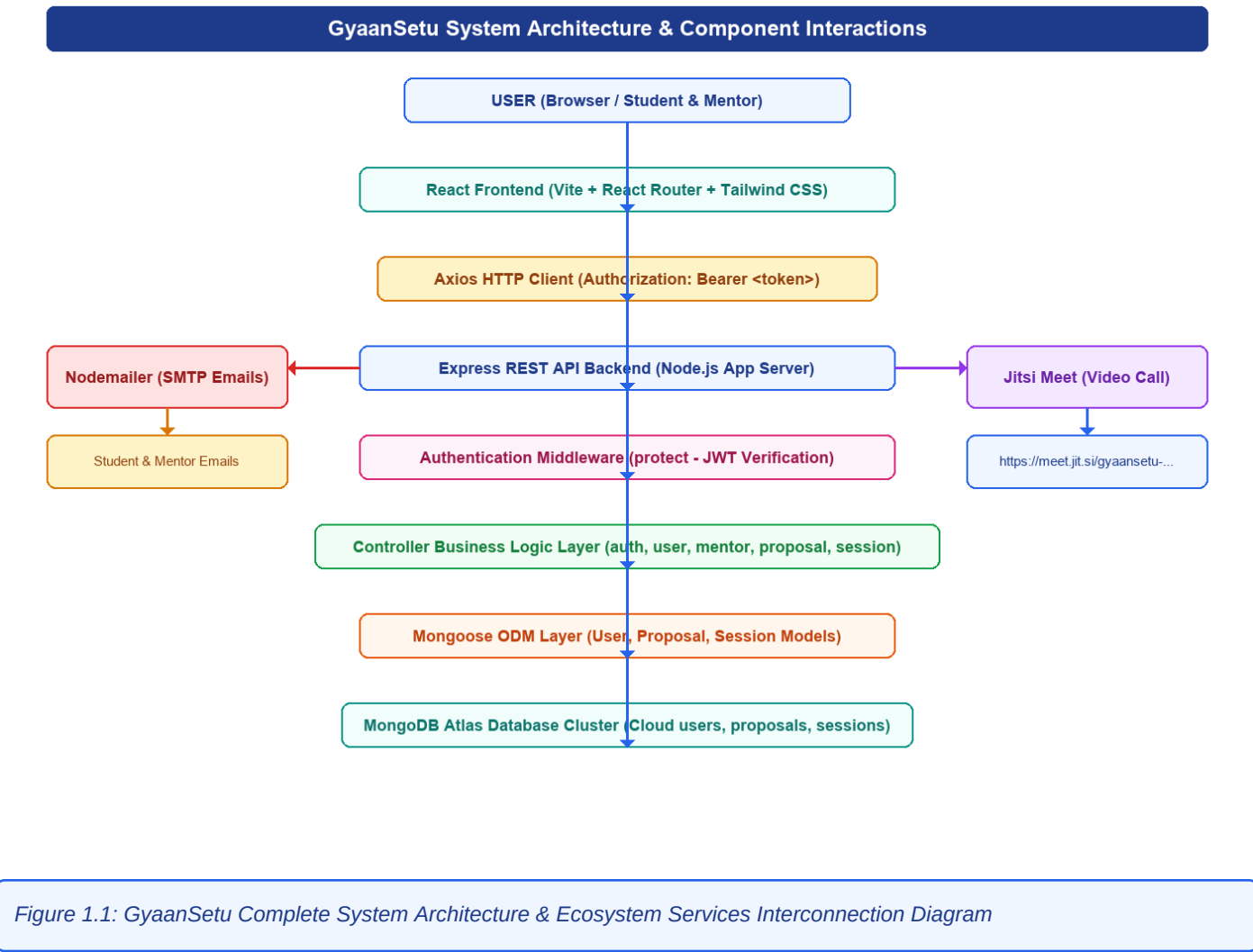
Author / Candidate:	Placement Candidate (Aditya)
Project Name:	GyaanSetu – Mentor-Student Doubt Solving Platform
Backend Stack:	Node.js, Express.js, MongoDB Atlas, Mongoose ODM, JWT, bcryptjs, Nodemailer
Database Design:	3 Collections (users, proposals, sessions) with Mongoose ObjectId References
Visual Diagrams Included:	System Architecture, Business Flow, Dual-Role Tree, Database ER, Entity Relationships, DB Lifecycle

Table of Contents

1. High-Level Project Architecture & Visual Diagram	11. Single Account Dual-Role Architecture & Diagram
2. 8-Layer Architecture Breakdown	12. Database Design & Visual ER Diagram
3. End-to-End Business Flowchart Diagram	13. Entity Relationship Visual Diagram
4. Server Entry Point (server.js)	14. Database State Machine Lifecycle Diagram
5. Database Connection & DNS (db.js)	15. Why Three Separate Collections?
6. Schema 1: User Model (User.js)	16. Database Document Flow Examples
7. Schema 2: Proposal Model (Proposal.js)	17. Mongoose Query Methods Line-by-Line
8. Schema 3: Session Model (Session.js)	18. Complete Request Lifecycle (9 Traces)
9. Authentication & Auth Middleware	19. Environment Variables Security
10. Controllers Code-by-Code Analysis	20. 60-Second Architecture & Database Speeches

1. High-Level Project Architecture & Visual Diagram

GyaanSetu is structured as a decoupled full-stack web application. The frontend React Single Page Application (SPA) interacts statelessly with the Express REST API backend over HTTP using JSON data formatting. Database operations are handled asynchronously by Mongoose ODM connected to MongoDB Atlas.



2. 8-Layer Architecture Breakdown

Layer	Implementing Files	Layer Responsibilities & Data Flow
1. Frontend Layer	<code>`client/src/pages/*`, `components/*`</code>	Captures user input, manages local state (`useState`), and renders clean responsive UI.
2. API / HTTP Layer	<code>`client/src/services/api.js`</code>	Axios instance appending `Authorization: Bearer ` to HTTP requests sent to backend.
3. Backend Route Layer	<code>`server/routes/*.js`</code>	Maps URL paths (`/api/proposals`) and HTTP methods (`POST`, `GET`, `PATCH`) to controllers.
4. Auth Middleware	<code>`server/middleware/authMiddleware.js`</code>	Intercepts requests, verifies JWT token signature, fetches user, and attaches `req.user`.
5. Controller Layer	<code>`server/controllers/*.js`</code>	Executes business rules, input validations, conflict checks, ownership authorization & responses.
6. Model / ODM Layer	<code>`server/models/*.js`</code>	Defines Mongoose schemas (`User`, `Proposal`, `Session`), data types, defaults, and references.
7. Database Layer	MongoDB Atlas Cluster	Persists JSON documents inside 3 collections (`users`, `proposals`, `sessions`).
8. External Services	Nodemailer, Jitsi Meet	Dispatches automated email notifications and generates virtual video room links.

3. End-to-End Business Flowchart Visual Diagram

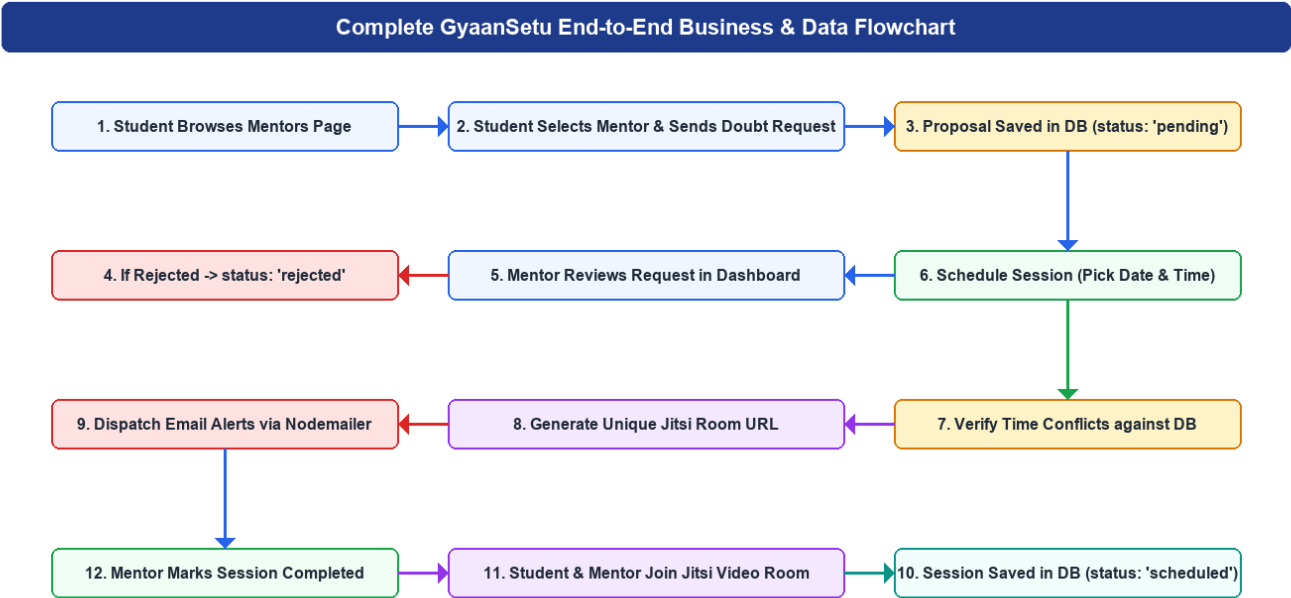


Figure 3.1: GyaanSetu End-to-End Doubt Solving & Session Scheduling Business Flowchart Diagram

4. Server Entry Point (server.js)

```
const express = require('express');
const cors = require('cors');
const dotenv = require('dotenv');
const connectDB = require('./config/db');

dotenv.config();
connectDB();

const app = express();
app.use(cors());
app.use(express.json());

app.use('/api/auth', require('./routes/authRoutes'));
app.use('/api/users', require('./routes/userRoutes'));
app.use('/api/mentors', require('./routes/mentorRoutes'));
app.use('/api/proposals', require('./routes/proposalRoutes'));
app.use('/api/sessions', require('./routes/sessionRoutes'));

app.get('/', (req, res) => res.json({ success: true, message: 'GyaanSetu Backend API is running smoothly.' }));

const PORT = process.env.PORT || 5000;
app.listen(PORT, () => console.log(`Server running on port ${PORT}`));
```

5. Database Connection & DNS Fallback (config/db.js)

```
const mongoose = require('mongoose');
const dns = require('dns');

try {
  dns.setServers(['8.8.8.8', '8.8.4.4']); // Override local DNS for SRV queries
} catch (err) {}

const connectDB = async () => {
  const targetUri = process.env.MONGO_URI || 'mongodb://127.0.0.1:27017/gyaansetu';
  try {
    const conn = await mongoose.connect(targetUri);
    console.log(`MongoDB Connected: ${conn.connection.host}`);
    return conn;
  } catch (error) {
    console.error(`Connection Error: ${error.message}`);
    throw error;
  }
};

module.exports = connectDB;
```

6. Schema 1: User Data Model (models/User.js)

Field Name	Data Type	Rules	Purpose in Application
`_id`	ObjectId	Auto Unique	MongoDB Primary Key
`name`	String	Required, Trimmed	Full user name
`email`	String	Required, Unique, Lowercase	Login handle & communication email
`password`	String	Required	Bcrypt hashed password
`isMentor`	Boolean	Default: `false`	Flag enabling mentor profile & incoming request controls
`skills`	Array of Strings	Default: `[]`	Technical tags (e.g. Java, React, SQL)
`createdAt`	Date	Default: `Date.now`	Account creation timestamp

7. Schema 2: Proposal Data Model (models/Proposal.js)

Field Name	Data Type	Reference / Rules	Purpose in Application
`_id`	ObjectId	Auto Unique	Proposal Primary Key
`student`	ObjectId	Ref: `User` (Required)	FK referencing student who submitted doubt
`mentor`	ObjectId	Ref: `User` (Required)	FK referencing mentor selected for help
`studentEmail`	String	Required	Snapshot of student email address
`mentorEmail`	String	Required	Snapshot of mentor email address
`doubt`	String	Required	Technical doubt description text
`status`	String	`['pending', 'accepted', 'rejected']`	State machine status (default: 'pending')
`createdAt`	Date	Default: `Date.now`	Timestamp when request was sent

8. Schema 3: Session Data Model (models/Session.js)

Field Name	Data Type	Reference / Rules	Purpose in Application
`_id`	ObjectId	Auto Unique	Session Primary Key
`proposal`	ObjectId	Ref: `Proposal` (Required)	FK referencing accepted proposal
`student`	ObjectId	Ref: `User` (Required)	FK referencing student participant
`mentor`	ObjectId	Ref: `User` (Required)	FK referencing mentor host
`scheduledTime`	Date	Required (Future Date)	Conflict-free scheduled session date & time
`meetingLink`	String	Required	Unique Jitsi Meet room URL (`https://meet.jit.si/...`)
`status`	String	`['scheduled', 'completed']`	Session status (default: 'scheduled')

11. Single Account — Student + Mentor Architecture & Diagram

Instead of maintaining split account types or separate login paths, GyaanSetu uses a **Single Account Architecture**. Every user is inherently a student. When a user clicks 'Become a Mentor', the API sets `isMentor = true` and updates `skills`. The exact same account immediately retains student capabilities while unlocking mentor features.

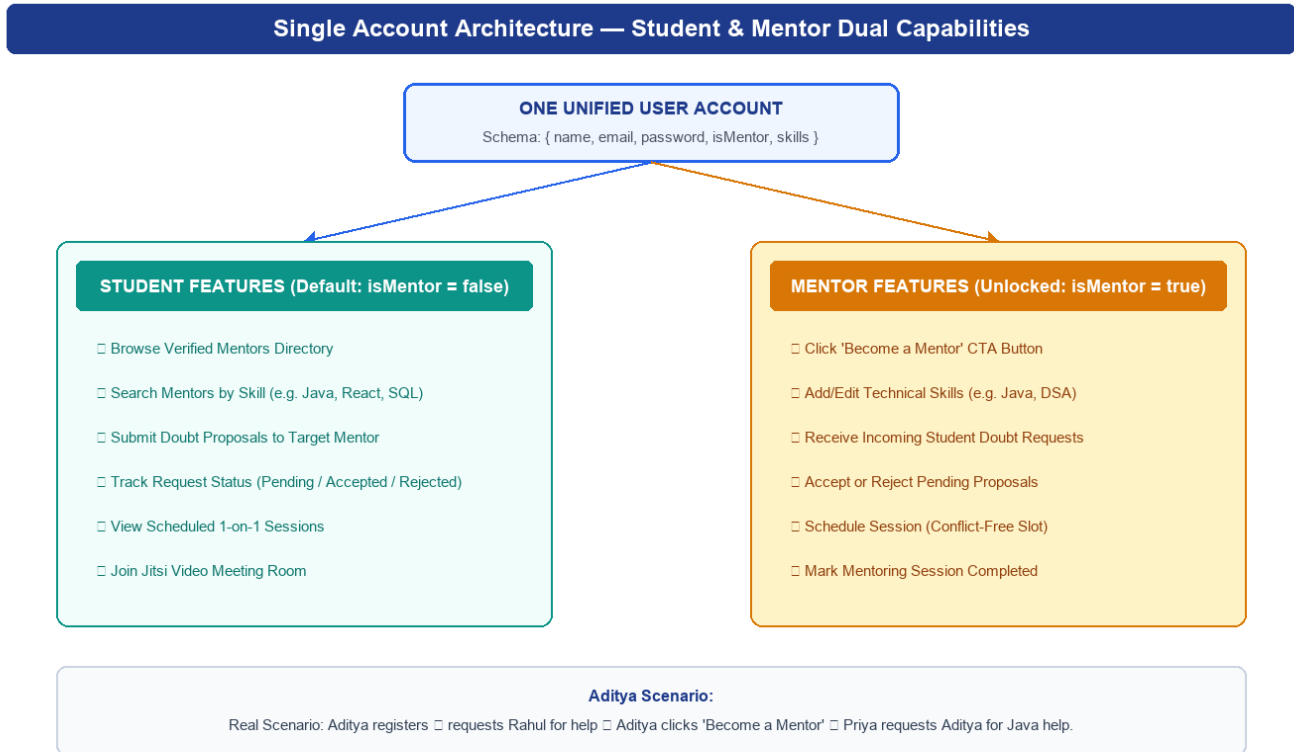
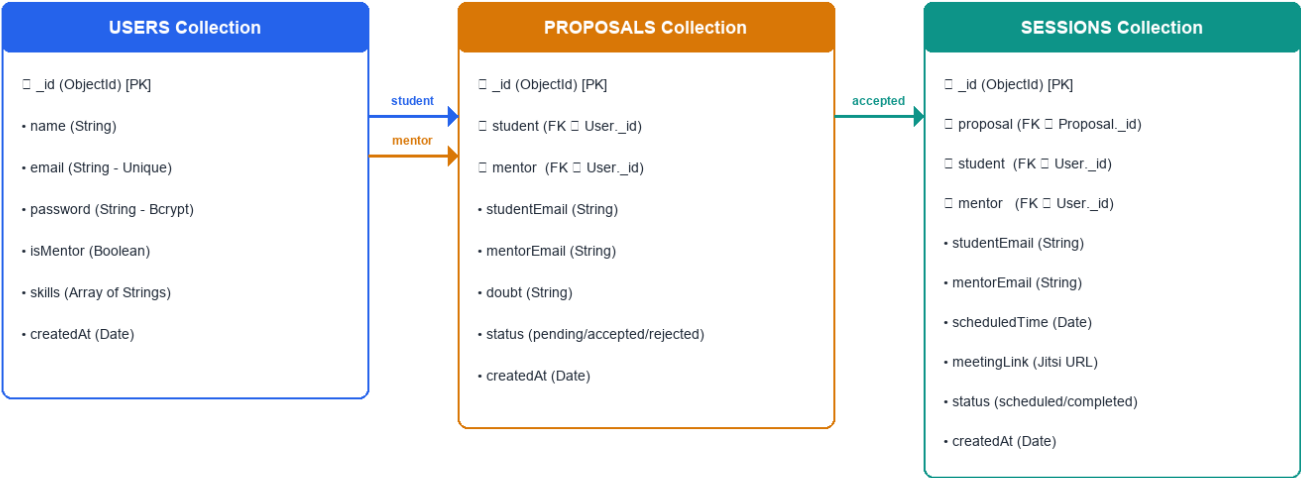


Figure 11.1: Single Account Dual Role Capabilities & Real Aditya Scenario Diagram

12. Database Design & Visual ER Diagram

The database contains 3 normalized collections inside the `gyaansetu` database on MongoDB Atlas:

GyaanSetu MongoDB Atlas ER Database Schema Diagram



Mongoose ObjectId References & Integrity Rules:

1. `proposal.student` & `proposal.mentor` store User ObjectIds (Mongoose populate('student mentor')).
2. `session.proposal` stores Proposal ObjectId (links accepted doubt to scheduled time & Jitsi room).
3. Decoupling rationale: Keeps user documents small and avoids 16MB document size limit.

Figure 12.1: GyaanSetu MongoDB Atlas ER Database Schema Diagram

13. Entity Relationship Visual Diagram

GyaanSetu Database Entity Relationships Summary

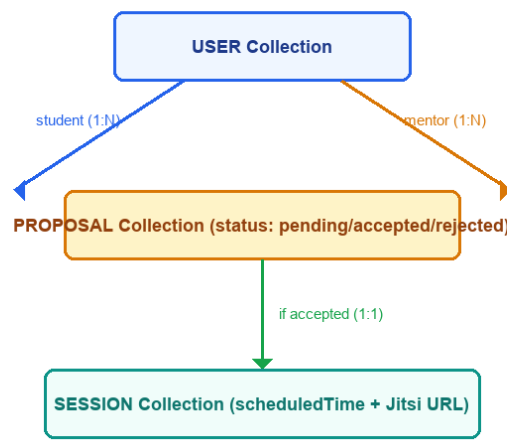


Figure 13.1: Database Collections Entity Relationships Flow Diagram

14. Database State Machine Lifecycle Diagram

Database State Machine Lifecycle Flowchart

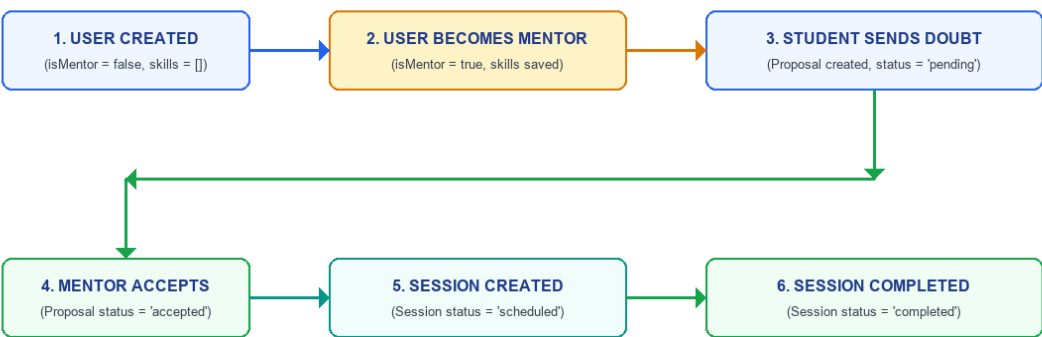


Figure 14.1: Database Documents State Machine Lifecycle Flowchart

15. Why Three Separate Collections?

- **Why not embed proposals in User?** Unbounded array anti-pattern! If a popular mentor receives hundreds of proposals, embedding arrays would exceed MongoDB's 16MB document size limit.
- **Why separate Session from Proposal?** Separation of concerns! A proposal represents an intent/request, while a session represents an active scheduled event with time conflict validation and meeting URLs.

16. Database Document Flow Examples

Scenario: From Doubt Request to Completed Session in MongoDB:

1. **Create Proposal:** ``Proposal.create({ student: studentId, mentor: mentorId, doubt: 'Help with SQL', status: 'pending' })``
2. **Accept Proposal:** ``Proposal.findByIdAndUpdate(proposalId, { status: 'accepted' })``
3. **Schedule Session:** ``Session.create({ proposal: proposalId, student, mentor, scheduledTime, meetingLink })``
4. **Complete Session:** ``Session.findByIdAndUpdate(sessionId, { status: 'completed' })``

17. Mongoose Query Methods Line-by-Line

```
// 1. Fetch all mentors for directory page
User.find({ isMentor: true }).select('-password').sort({ createdAt: -1 });

// 2. Prevent duplicate active requests between same student & mentor
Proposal.findOne({ student: studentId, mentor: mentorId, status: 'pending' });

// 3. Time slot conflict query for scheduling
Session.findOne({
  status: 'scheduled',
  $or: [
    { mentor, scheduledTime: { $gte: startTimeWindow, $lte: endTimeWindow } },
    { student, scheduledTime: { $gte: startTimeWindow, $lte: endTimeWindow } }
  ]
});

// 4. Data enrichment using populate
Proposal.find({ student: req.user._id }).populate('mentor', 'name email skills');
```

18. Complete Request Lifecycle (9 Traces)

Trace 1: User Registration → POST `/api/auth/register` → check existing email → bcrypt hash → `User.create` → issue JWT.

Trace 2: User Login → POST `/api/auth/login` → find email → `bcrypt.compare` → issue JWT.

Trace 3: Get Mentors → GET `/api/mentors` → query `isMentor: true` → return JSON directory.

Trace 4: Send Doubt → POST `/api/proposals` → check self-request & duplicate → `Proposal.create`.

Trace 5: Accept Request → PATCH `/api/proposals/:id/accept` → verify mentor ownership → set status `accepted`.

Trace 6: Become Mentor → PATCH `/api/users/become-mentor` → parse skills array → set `isMentor = true`.

Trace 7: Schedule Session → POST `/api/sessions` → check time conflict → generate Jitsi link → send email.

Trace 8: Get Sessions → GET `/api/sessions/my` → query `\$or: [{ student }, { mentor }]` → return sessions.

Trace 9: Complete Session → PATCH `/api/sessions/:id/complete` → verify mentor ownership → set status `completed`.

19. Environment Variables Security

Variable Name	Purpose	Security Risk if Exposed
`PORT`	HTTP Server Port (default 5000)	Low (public network configuration)
`MONGO_URI`	MongoDB Atlas connection string	CRITICAL: Exposes database read/write access
`JWT_SECRET`	Secret key for signing auth tokens	CRITICAL: Allows attackers to forge fake auth tokens
`EMAIL_USER`	Nodemailer SMTP email address	Medium: Spam risk
`EMAIL_PASS`	SMTP application password	HIGH: Unauthorized email sending

20. 60-Second Architecture & Database Speeches

EXPLAIN MY PROJECT ARCHITECTURE IN 60 SECONDS:

"GyaanSetu is built on a decoupled MERN architecture. The frontend is a React 18 SPA compiled with Vite and styled using Tailwind CSS. It communicates statelessly with an Express REST API running on Node.js using Axios. Requests carry JWT tokens in the `Authorization: Bearer` header, which custom `protect` middleware verifies to populate `req.user`. Controllers execute business rules, validation, and conflict checks before persisting data into MongoDB Atlas via Mongoose ODM. We also integrate Nodemailer for session email alerts and Jitsi Meet for automated 1-on-1 video call rooms. The architecture enforces a single-account model where any student can upgrade to a mentor without separate logins."

EXPLAIN MY DATABASE DESIGN IN 60 SECONDS:

"Our MongoDB Atlas database contains 3 normalized collections: `users`, `proposals`, and `sessions`. The `users` collection stores user details, hashed passwords, `isMentor` boolean flag, and skills array. The `proposals` collection links a student `ObjectId` to a mentor `ObjectId` along with the doubt description and status state machine (`pending`, `accepted`, `rejected`). The `sessions` collection links an accepted proposal to a scheduled time, status (`scheduled`, `completed`), and a generated Jitsi meeting URL. We use Mongoose `ObjectId` references to perform populated queries while avoiding unbounded array growth inside user documents."